Indian Institute of Technology Jodhpur - Rajasthan

School of Data Science and Artificial Intelligence

Assignment 3: Movie Recommendations

Machine Learning with Big Data

Submitted to: Dr. Dip Shankar Banerjee

Associate Professor

School of Data Science and Artificial Intelligence

Indian Institute of Technology Jodhpur

Submitted by: Purushothaman S

Roll Number: M25DE1033

Email id: m25de1033@iitj.ac.in

Course: M.Tech. – DATA ENGINEERING - 2nd Year

Indian Institute of Technology Jodhpur

Abstract

Recommender systems play an important role in modern digital platforms such as **Netflix, Amazon, and YouTube**, where large numbers of items must be suggested to users based on their preferences. The goal of this project is to design and evaluate multiple recommendation techniques using the **MovieLens dataset**, which contains user ratings and movie metadata.

This project explores different recommendation approaches including **Content-Based Filtering, Collaborative Filtering, Matrix Factorization, Hybrid Models, Neural Network based recommenders, and Reinforcement Learning methods**. Each technique is implemented and evaluated using appropriate performance metrics such as **RMSE, Precision@K, and Recall@K**.

The study also examines the advantages and limitations of each recommendation method and investigates how combining multiple techniques can improve recommendation accuracy. Finally, the project explores **explainability techniques** to make recommendation decisions more transparent and interpretable.

The results demonstrate that hybrid and learning-based recommendation systems provide **better personalization and improved prediction accuracy** compared to traditional recommendation approaches.

Table of Contents

1. Abstract
2. Introduction
3. Dataset Description
4. Part 1 – Content Based Filtering
    - Task 1 – TF-IDF Based Recommendation
    - Task 2 – User Profile Based Recommendation
5. Part 2 – Collaborative Filtering
    - Task 3 – User Based Collaborative Filtering
    - Task 4 – Item Based Collaborative Filtering
6. Part 3 – Matrix Factorization
    - Task 5 – Singular Value Decomposition
    - Task 6 – Matrix Factorization using Surprise Library
7. Part 4 – Hybrid Recommendation System
8. Part 5 – Learning Based Recommendation
    - Task 8 – Neural Network Based Recommender
    - Task 9 – Reinforcement Learning Based Recommender
9. Part 6 – Explainability in Recommender Systems
10. Results and Discussion
11. Conclusion
12. References

1. Introduction

Recommender systems are intelligent systems designed to **predict user preferences and suggest relevant items** based on historical interactions. These systems are widely used in online platforms to help users discover new products, movies, music, or services that match their interests.

The rapid growth of digital content has created a **data overload problem**, where users struggle to find relevant information. Recommender systems solve this problem by analyzing **user behaviour, item characteristics, and historical interaction data** to provide personalized suggestions.

In this project, several recommendation algorithms are implemented using the **MovieLens dataset**, which is widely used as a benchmark dataset in recommendation system research. The dataset contains **user ratings, movie metadata, and genre information**, allowing multiple recommendation approaches to be explored.

The primary objective of this project is to implement and evaluate different recommendation techniques including:

- **Content-Based Filtering**
- **Collaborative Filtering**
- **Matrix Factorization**
- **Hybrid Recommendation Systems**
- **Neural Network Based Recommendations**
- **Reinforcement Learning Based Recommendations**
- **Explainable AI methods for recommendations**

Each technique is evaluated using different performance metrics to understand their strengths and limitations. The study also investigates how advanced techniques such as **hybrid models and neural networks can improve recommendation quality** compared to traditional approaches.

2. Dataset Description

The dataset used in this project is the **MovieLens dataset**, which is a widely used dataset for research in recommender systems.

The dataset contains information about:

- **User ratings for movies**

- **Movie metadata such as genres**
- **User interaction history**

This dataset enables the implementation of both **content-based recommendation techniques and collaborative filtering algorithms**.

Important components of the dataset include:

- **movies.csv** – Contains movie titles and genres
- **ratings.csv** – Contains user ratings for movies
- **links.csv** – Contains mapping between movie IDs

These datasets provide the required information for building recommendation models.

```
============================================================
Movie Recommendation Project
============================================================

Dataset loaded successfully.

Movies shape  : (9742, 3)
Ratings shape : (100836, 4)
Tags shape    : (3683, 4)
Links shape   : (9742, 3)

============================================================
Preprocessing Data
============================================================
Processed dataset saved to processed/merged_dataset.csv
Processed dataset saved to: /Users/niranjan/Downloads/MovieRecommendation/data/processed/processed_movies_ratings.csv
```

3. Part 1 – Content Based Filtering

Content-based filtering recommends items based on the **similarity between item features**. In this approach, the system recommends movies that are similar to the movies the user has liked previously.

Task 1 – TF-IDF Based Recommendation

In this task, a **TF-IDF based recommendation system** was implemented to recommend movies based on their genre similarity.

TF-IDF (Term Frequency – Inverse Document Frequency) is a statistical technique used to measure the importance of words in a document relative to a collection of documents. In the context of this project, movie genres are treated as textual features.

The steps involved in this task include:

1. Loading the MovieLens dataset.
2. Extracting movie genres as textual features.

3. Applying **TF-IDF vectorization** to convert genre information into numerical vectors.
4. Computing **cosine similarity between movie vectors**.
5. Creating a recommendation function that returns the **top N most similar movies** for a given movie.

This approach allows the system to recommend movies that share similar genres and characteristics.

Code Implementation

```
import numpy as np
import pandas as pd
from scipy.sparse import csr_matrix
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def build_tfidf_similarity(movies: pd.DataFrame):
    """
    Build TF-IDF matrix from movie genres and compute cosine similarity.
    """
    movies = movies.copy()

    # Replace missing genres if any
    movies["genres"] = movies["genres"].fillna("")

    # Convert pipe-separated genres into space-separated text
    movies["genres_text"] = movies["genres"].str.replace("|", " ", regex=False)

    tfidf = TfidfVectorizer(stop_words="english")
    tfidf_matrix = tfidf.fit_transform(movies["genres_text"])
```

```
    similarity_matrix = cosine_similarity(tfidf_matrix, tfidf_matrix)

    return movies, tfidf_matrix, similarity_matrix

def get_content_recommendations(
    title: str,
    movies: pd.DataFrame,
    similarity_matrix,
    top_n: int = 10
) -> pd.DataFrame:
    """
    Recommend similar movies based on genre TF-IDF similarity.
    """
    matches = movies[movies["title"].str.lower() == title.lower()]

    if matches.empty:
        raise ValueError(f"Movie '{title}' not found in dataset.")

    movie_index = matches.index[0]

    similarity_scores = list(enumerate(similarity_matrix[movie_index]))
    similarity_scores = sorted(similarity_scores, key=lambda x: x[1], reverse=True)

    # Exclude the same movie itself
    similarity_scores = similarity_scores[1: top_n + 1]

    movie_indices = [i[0] for i in similarity_scores]
```

```
    scores = [i[1] for i in similarity_scores]

    recommendations = movies.iloc[movie_indices][["movieId", "title",
"genres"]].copy()
    recommendations["similarity_score"] = scores

    return recommendations.reset_index(drop=True)

def build_user_profile(
    user_id: int,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    tfidf_matrix
):
    """
    Build a user profile as a weighted average of TF-IDF movie vectors,
    weighted by the user's ratings.
    """
    user_ratings = ratings[ratings["userId"] == user_id].copy()

    if user_ratings.empty:
        raise ValueError(f"User {user_id} not found in ratings data.")

    # Merge ratings with movie indices
    movie_index_map = pd.Series(movies.index, index=movies["movieId"]).to_dict()
    user_ratings["movie_index"] = user_ratings["movieId"].map(movie_index_map)
    user_ratings = user_ratings.dropna(subset=["movie_index"]).copy()
    user_ratings["movie_index"] = user_ratings["movie_index"].astype(int)
```

```
    if user_ratings.empty:
        raise ValueError(f"No matching rated movies found for user {user_id}.")

    rated_indices = user_ratings["movie_index"].values
    weights = user_ratings["rating"].values.reshape(-1, 1)

    rated_movie_vectors = tfidf_matrix[rated_indices]

    # Weighted sum of rated movie vectors
    weighted_profile = rated_movie_vectors.multiply(weights).sum(axis=0)

    # Normalize by total rating sum
    total_rating = user_ratings["rating"].sum()
    user_profile = weighted_profile / total_rating

    return csr_matrix(user_profile), user_ratings

def get_user_profile_recommendations(
    user_id: int,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    tfidf_matrix,
    top_n: int = 10
) -> pd.DataFrame:
    """
    Recommend movies to a user based on cosine similarity between
    the user profile and all movie TF-IDF vectors.
```

```
    """
    user_profile, user_ratings = build_user_profile(
        user_id=user_id,
        ratings=ratings,
        movies=movies,
        tfidf_matrix=tfidf_matrix,
    )

    similarity_scores = cosine_similarity(user_profile, tfidf_matrix).flatten()

    recommendations = movies.copy()
    recommendations["similarity_score"] = similarity_scores

    # Remove already-rated movies
    rated_movie_ids = set(user_ratings["movieId"].tolist())
    recommendations =
recommendations[~recommendations["movieId"].isin(rated_movie_ids)]

    recommendations = recommendations.sort_values(
        by="similarity_score", ascending=False
    ).head(top_n)

    return recommendations[["movieId", "title", "genres",
"similarity_score"]].reset_index(drop=True)

def precision_recall_at_k_content(
    user_id: int,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
```

```
    tfidf_matrix,
    k: int = 10,
    relevant_threshold: float = 4.0
):
    """
    Compute Precision@K and Recall@K for a user-profile recommender.

    Relevant movies = movies the user rated >= relevant_threshold.
    Ground truth is derived from the same user's highly rated movies.
    This is a simple educational evaluation aligned with the assignment.
    """
    user_all_ratings = ratings[ratings["userId"] == user_id].copy()

    if user_all_ratings.empty:
        raise ValueError(f"User {user_id} not found in ratings data.")

    relevant_movies = set(
        user_all_ratings[user_all_ratings["rating"] >=
relevant_threshold]["movieId"].tolist()
    )

    if len(relevant_movies) == 0:
        return 0.0, 0.0

    recommendations = get_user_profile_recommendations(
        user_id=user_id,
        ratings=ratings,
        movies=movies,
```

```
    tfidf_matrix=tfidf_matrix,
    top_n=k
)

recommended_movies = set(recommendations["movieId"].tolist())
true_positives = len(recommended_movies.intersection(relevant_movies))

precision = true_positives / k if k > 0 else 0.0
recall = true_positives / len(relevant_movies) if len(relevant_movies) > 0 else 0.0

return precision, recall
```

Output

```
============================================================
Content-Based Recommendation
============================================================
   movieId                                              title                                       genres  similarity_score
0     2294                                          Antz (1998)  Adventure|Animation|Children|Comedy|Fantasy               1.0
1     3114                                     Toy Story 2 (1999)  Adventure|Animation|Children|Comedy|Fantasy               1.0
2     3754     Adventures of Rocky and Bullwinkle, The (2000)  Adventure|Animation|Children|Comedy|Fantasy               1.0
3     4016                   Emperor's New Groove, The (2000)  Adventure|Animation|Children|Comedy|Fantasy               1.0
4     4886                              Monsters, Inc. (2001)  Adventure|Animation|Children|Comedy|Fantasy               1.0
5    45074                                   Wild, The (2006)  Adventure|Animation|Children|Comedy|Fantasy               1.0
6    53121                              Shrek the Third (2007)  Adventure|Animation|Children|Comedy|Fantasy               1.0
7    65577                     Tale of Despereaux, The (2008)  Adventure|Animation|Children|Comedy|Fantasy               1.0
8    91355  Asterix and the Vikings (Astérix et les Viking...  Adventure|Animation|Children|Comedy|Fantasy               1.0
9   103755                                         Turbo (2013)  Adventure|Animation|Children|Comedy|Fantasy               1.0
```

Task 2 – User Profile-Based Content Recommender

In this task, a **user-profile-based recommendation system** was implemented to personalize movie recommendations based on user preferences.

Instead of recommending movies only based on item similarity, the system constructs a **user profile vector** that represents the user's overall preferences. The user profile is computed as a **weighted average of the TF-IDF vectors of the movies rated by the user**, where the weights correspond to the ratings provided by the user.

The user profile is calculated using the following equation:

Pu = Σ (rating × movie vector) / Σ (ratings)

Once the user profile is constructed, the system calculates the **cosine similarity between the user profile and movie vectors** to determine how relevant each movie is for the user.

Movies with the highest similarity scores are recommended to the user.

This approach improves personalization because it captures the **individual taste of each user**.

Code Implementation

```
import numpy as np
import pandas as pd
from scipy.sparse import csr_matrix
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def build_tfidf_similarity(movies: pd.DataFrame):
    """
    Build TF-IDF matrix from movie genres and compute cosine similarity.
    """
    movies = movies.copy()

    # Replace missing genres if any
    movies["genres"] = movies["genres"].fillna("")

    # Convert pipe-separated genres into space-separated text
    movies["genres_text"] = movies["genres"].str.replace("|", " ", regex=False)

    tfidf = TfidfVectorizer(stop_words="english")
    tfidf_matrix = tfidf.fit_transform(movies["genres_text"])

    similarity_matrix = cosine_similarity(tfidf_matrix, tfidf_matrix)
```

```
    return movies, tfidf_matrix, similarity_matrix


def get_content_recommendations(
    title: str,
    movies: pd.DataFrame,
    similarity_matrix,
    top_n: int = 10
) -> pd.DataFrame:
    """
    Recommend similar movies based on genre TF-IDF similarity.
    """
    matches = movies[movies["title"].str.lower() == title.lower()]

    if matches.empty:
        raise ValueError(f"Movie '{title}' not found in dataset.")

    movie_index = matches.index[0]

    similarity_scores = list(enumerate(similarity_matrix[movie_index]))
    similarity_scores = sorted(similarity_scores, key=lambda x: x[1], reverse=True)

    # Exclude the same movie itself
    similarity_scores = similarity_scores[1: top_n + 1]

    movie_indices = [i[0] for i in similarity_scores]
    scores = [i[1] for i in similarity_scores]
```

```
    recommendations = movies.iloc[movie_indices][["movieId", "title",
"genres"]].copy()
    recommendations["similarity_score"] = scores

    return recommendations.reset_index(drop=True)

def build_user_profile(
    user_id: int,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    tfidf_matrix
):
    """
    Build a user profile as a weighted average of TF-IDF movie vectors,
    weighted by the user's ratings.
    """
    user_ratings = ratings[ratings["userId"] == user_id].copy()

    if user_ratings.empty:
        raise ValueError(f"User {user_id} not found in ratings data.")

    # Merge ratings with movie indices
    movie_index_map = pd.Series(movies.index, index=movies["movieId"]).to_dict()
    user_ratings["movie_index"] = user_ratings["movieId"].map(movie_index_map)
    user_ratings = user_ratings.dropna(subset=["movie_index"]).copy()
    user_ratings["movie_index"] = user_ratings["movie_index"].astype(int)

    if user_ratings.empty:
```

```
        raise ValueError(f"No matching rated movies found for user {user_id}.")

    rated_indices = user_ratings["movie_index"].values
    weights = user_ratings["rating"].values.reshape(-1, 1)

    rated_movie_vectors = tfidf_matrix[rated_indices]

    # Weighted sum of rated movie vectors
    weighted_profile = rated_movie_vectors.multiply(weights).sum(axis=0)

    # Normalize by total rating sum
    total_rating = user_ratings["rating"].sum()
    user_profile = weighted_profile / total_rating

    return csr_matrix(user_profile), user_ratings

def get_user_profile_recommendations(
    user_id: int,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    tfidf_matrix,
    top_n: int = 10
) -> pd.DataFrame:
    """
    Recommend movies to a user based on cosine similarity between
    the user profile and all movie TF-IDF vectors.
    """
    user_profile, user_ratings = build_user_profile(
```

```
        user_id=user_id,
        ratings=ratings,
        movies=movies,
        tfidf_matrix=tfidf_matrix,
    )

    similarity_scores = cosine_similarity(user_profile, tfidf_matrix).flatten()

    recommendations = movies.copy()
    recommendations["similarity_score"] = similarity_scores

    # Remove already-rated movies
    rated_movie_ids = set(user_ratings["movieId"].tolist())
    recommendations =
recommendations[~recommendations["movieId"].isin(rated_movie_ids)]

    recommendations = recommendations.sort_values(
        by="similarity_score", ascending=False
    ).head(top_n)

    return recommendations[["movieId", "title", "genres",
"similarity_score"]].reset_index(drop=True)

def precision_recall_at_k_content(
    user_id: int,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    tfidf_matrix,
    k: int = 10,
```

```
    relevant_threshold: float = 4.0
):
    """
    Compute Precision@K and Recall@K for a user-profile recommender.

    Relevant movies = movies the user rated >= relevant_threshold.
    Ground truth is derived from the same user's highly rated movies.
    This is a simple educational evaluation aligned with the assignment.
    """
    user_all_ratings = ratings[ratings["userId"] == user_id].copy()

    if user_all_ratings.empty:
        raise ValueError(f"User {user_id} not found in ratings data.")

    relevant_movies = set(
        user_all_ratings[user_all_ratings["rating"] >=
relevant_threshold]["movieId"].tolist()
    )

    if len(relevant_movies) == 0:
        return 0.0, 0.0

    recommendations = get_user_profile_recommendations(
        user_id=user_id,
        ratings=ratings,
        movies=movies,
        tfidf_matrix=tfidf_matrix,
        top_n=k
```

```
)

recommended_movies = set(recommendations["movieId"].tolist())
true_positives = len(recommended_movies.intersection(relevant_movies))

precision = true_positives / k if k > 0 else 0.0
recall = true_positives / len(relevant_movies) if len(relevant_movies) > 0 else 0.0

return precision, recall
```

Output

```
==========================================================
User Profile Content Recommendation
==========================================================
   movieId                                title                                          genres  similarity_score
0   117646          Dragonheart 2: A New Beginning (2000)   Action|Adventure|Comedy|Drama|Fantasy|Thriller          0.791026
1    55116                     Hunting Party, The (2007)          Action|Adventure|Comedy|Drama|Thriller          0.773645
2     5657                                Flashback (1990)              Action|Adventure|Comedy|Crime|Drama          0.773089
3     6990                 The Great Train Robbery (1978)              Action|Adventure|Comedy|Crime|Drama          0.773089
4    26184  Diamond Arm, The (Brilliantovaya ruka) (1968)          Action|Adventure|Comedy|Crime|Thriller          0.760503
5     8968                       After the Sunset (2004)          Action|Adventure|Comedy|Crime|Thriller          0.760503
6     6503         Charlie's Angels: Full Throttle (2003)          Action|Adventure|Comedy|Crime|Thriller          0.760503
7    80219                                Machete (2010)          Action|Adventure|Comedy|Crime|Thriller          0.760503
8   164226                           Maximum Ride (2016)  Action|Adventure|Comedy|Fantasy|Sci-Fi|Thriller          0.759670
9     4956                          Stunt Man, The (1980)   Action|Adventure|Comedy|Drama|Romance|Thriller          0.753983
```

4. Part 2 – Collaborative Filtering

Collaborative filtering recommends items based on the **preferences of similar users or similar items**.

Unlike content-based filtering, collaborative filtering **does not rely on item features** but instead uses user interaction patterns.

Task 3 – User Based Collaborative Filtering

User-based collaborative filtering works on the assumption that **users with similar preferences will like similar items**.

The steps involved include:

1. Constructing a **user-item rating matrix**.
2. Computing **similarity between users** using cosine similarity or Pearson correlation.
3. Selecting the **top K similar users**.

4. Predicting ratings for unseen movies using the **weighted average of ratings from similar users**.
5. Generating recommendations based on predicted ratings.

The system is evaluated using metrics such as **Precision@K and Recall@K**.

Task 4 – Item Based Collaborative Filtering

Item-based collaborative filtering recommends movies that are **similar to the movies a user has already liked**.

The process includes:

- Computing similarity between movies using **user rating patterns**
- Predicting ratings based on **similar items**
- Ranking movies according to predicted ratings

This method is often **more scalable than user-based filtering** because item similarity can be precomputed.

Code Implementation

```
import numpy as np

import pandas as pd

from sklearn.metrics.pairwise import cosine_similarity

from sklearn.metrics import mean_squared_error


def build_user_item_matrix(ratings: pd.DataFrame) -> pd.DataFrame:

    """

    Create user-item rating matrix.

    Rows = users, columns = movies, values = ratings.

    """

    user_item = ratings.pivot_table(

        index="userId",

        columns="movieId",
```

```
        values="rating"
    )
    return user_item

def compute_user_similarity(user_item_matrix: pd.DataFrame) -> pd.DataFrame:
    """
    Compute user-user cosine similarity.
    Missing ratings are filled with 0 for similarity calculation.
    """
    filled_matrix = user_item_matrix.fillna(0)
    similarity = cosine_similarity(filled_matrix)

    user_similarity_df = pd.DataFrame(
        similarity,
        index=user_item_matrix.index,
        columns=user_item_matrix.index
    )
    return user_similarity_df

def compute_item_similarity(user_item_matrix: pd.DataFrame) -> pd.DataFrame:
    """
    Compute item-item cosine similarity.
    Missing ratings are filled with 0 for similarity calculation.
    """
    filled_matrix = user_item_matrix.fillna(0).T
    similarity = cosine_similarity(filled_matrix)

    item_similarity_df = pd.DataFrame(
```

```
        similarity,
        index=filled_matrix.index,
        columns=filled_matrix.index
    )
    return item_similarity_df

def predict_rating_user_based(
    target_user: int,
    movie_id: int,
    user_item_matrix: pd.DataFrame,
    user_similarity: pd.DataFrame,
    k: int = 20
) -> float:
    """
    Predict a user's rating for a movie using top-K similar users.
    """
    if movie_id not in user_item_matrix.columns:
        return np.nan

    if target_user not in user_item_matrix.index:
        return np.nan

    movie_ratings = user_item_matrix[movie_id].dropna()

    if movie_ratings.empty:
        return np.nan

    sims = user_similarity.loc[target_user, movie_ratings.index]
```

```
    if target_user in sims.index:
        sims = sims.drop(target_user, errors="ignore")
        movie_ratings = movie_ratings.drop(target_user, errors="ignore")

    if sims.empty:
        return np.nan

    top_k_users = sims.sort_values(ascending=False).head(k)
    neighbor_ratings = movie_ratings.loc[top_k_users.index]

    denominator = np.abs(top_k_users).sum()
    if denominator == 0:
        return np.nan

    predicted_rating = np.dot(top_k_users.values, neighbor_ratings.values) /
denominator
    return float(predicted_rating)

def predict_rating_item_based(
    target_user: int,
    movie_id: int,
    user_item_matrix: pd.DataFrame,
    item_similarity: pd.DataFrame,
    k: int = 20
) -> float:
    """
    Predict a user's rating for a movie using top-K similar items
```

```
that the user has already rated.
"""
if target_user not in user_item_matrix.index:
    return np.nan

if movie_id not in user_item_matrix.columns:
    return np.nan

user_ratings = user_item_matrix.loc[target_user].dropna()

if user_ratings.empty:
    return np.nan

rated_movie_ids = user_ratings.index.tolist()

if movie_id not in item_similarity.index:
    return np.nan

sims = item_similarity.loc[movie_id, rated_movie_ids]

if sims.empty:
    return np.nan

top_k_items = sims.sort_values(ascending=False).head(k)
neighbor_ratings = user_ratings.loc[top_k_items.index]

denominator = np.abs(top_k_items).sum()
if denominator == 0:
```

```
        return np.nan

    predicted_rating = np.dot(top_k_items.values, neighbor_ratings.values) /
denominator
    return float(predicted_rating)

def recommend_user_based(
    target_user: int,
    user_item_matrix: pd.DataFrame,
    user_similarity: pd.DataFrame,
    movies: pd.DataFrame,
    k: int = 20,
    top_n: int = 10
) -> pd.DataFrame:
    """
    Recommend top-N unseen movies for a target user.
    """
    if target_user not in user_item_matrix.index:
        raise ValueError(f"User {target_user} not found.")

    user_rated_movies = user_item_matrix.loc[target_user].dropna().index.tolist()
    unseen_movies = [m for m in user_item_matrix.columns if m not in
user_rated_movies]

    predictions = []
    for movie_id in unseen_movies:
        pred = predict_rating_user_based(
            target_user=target_user,
            movie_id=movie_id,
```

```
            user_item_matrix=user_item_matrix,
            user_similarity=user_similarity,
            k=k
        )
        if not np.isnan(pred):
            predictions.append((movie_id, pred))

    pred_df = pd.DataFrame(predictions, columns=["movieId", "predicted_rating"])
    pred_df = pred_df.sort_values(by="predicted_rating",
ascending=False).head(top_n)

    result = pred_df.merge(movies, on="movieId", how="left")
    return result[["movieId", "title", "genres",
"predicted_rating"]].reset_index(drop=True)

def recommend_item_based(
    target_user: int,
    user_item_matrix: pd.DataFrame,
    item_similarity: pd.DataFrame,
    movies: pd.DataFrame,
    k: int = 20,
    top_n: int = 10
) -> pd.DataFrame:
    """
    Recommend top-N unseen movies for a target user using item-based CF.
    """
    if target_user not in user_item_matrix.index:
        raise ValueError(f"User {target_user} not found.")
```

```
    user_rated_movies = user_item_matrix.loc[target_user].dropna().index.tolist()
    unseen_movies = [m for m in user_item_matrix.columns if m not in
user_rated_movies]

    predictions = []
    for movie_id in unseen_movies:
        pred = predict_rating_item_based(
            target_user=target_user,
            movie_id=movie_id,
            user_item_matrix=user_item_matrix,
            item_similarity=item_similarity,
            k=k
        )
        if not np.isnan(pred):
            predictions.append((movie_id, pred))

    pred_df = pd.DataFrame(predictions, columns=["movieId", "predicted_rating"])
    pred_df = pred_df.sort_values(by="predicted_rating",
ascending=False).head(top_n)

    result = pred_df.merge(movies, on="movieId", how="left")
    return result[["movieId", "title", "genres",
"predicted_rating"]].reset_index(drop=True)

def evaluate_rmse_user_based(
    ratings: pd.DataFrame,
    user_item_matrix: pd.DataFrame,
    user_similarity: pd.DataFrame,
    k: int = 20,
```

```
    sample_size: int = 1000
) -> float:
    """
    Simple RMSE evaluation on a sample of known ratings.
    """
    sample = ratings.sample(n=min(sample_size, len(ratings)), random_state=42)

    y_true = []
    y_pred = []

    for _, row in sample.iterrows():
        user_id = row["userId"]
        movie_id = row["movieId"]
        true_rating = row["rating"]

        pred = predict_rating_user_based(
            target_user=user_id,
            movie_id=movie_id,
            user_item_matrix=user_item_matrix,
            user_similarity=user_similarity,
            k=k
        )

        if not np.isnan(pred):
            y_true.append(true_rating)
            y_pred.append(pred)

    if len(y_true) == 0:
```

```
        return np.nan

    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    return float(rmse)

def evaluate_rmse_item_based(
    ratings: pd.DataFrame,
    user_item_matrix: pd.DataFrame,
    item_similarity: pd.DataFrame,
    k: int = 20,
    sample_size: int = 1000
) -> float:
    """
    Simple RMSE evaluation on a sample of known ratings for item-based CF.
    """
    sample = ratings.sample(n=min(sample_size, len(ratings)), random_state=42)

    y_true = []
    y_pred = []

    for _, row in sample.iterrows():
        user_id = row["userId"]
        movie_id = row["movieId"]
        true_rating = row["rating"]

        pred = predict_rating_item_based(
            target_user=user_id,
            movie_id=movie_id,
```

```
            user_item_matrix=user_item_matrix,
            item_similarity=item_similarity,
            k=k
        )

        if not np.isnan(pred):
            y_true.append(true_rating)
            y_pred.append(pred)

    if len(y_true) == 0:
        return np.nan

    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    return float(rmse)

def precision_recall_at_k_user_based(
    target_user: int,
    ratings: pd.DataFrame,
    user_item_matrix: pd.DataFrame,
    user_similarity: pd.DataFrame,
    movies: pd.DataFrame,
    k_neighbors: int = 20,
    top_k: int = 10,
    relevant_threshold: float = 4.0
):
    """
    Compute Precision@K and Recall@K for user-based CF.
    """
```

```
    user_true = ratings[ratings["userId"] == target_user]
    relevant_items = set(user_true[user_true["rating"] >=
relevant_threshold]["movieId"].tolist())

    if len(relevant_items) == 0:
        return 0.0, 0.0

    recs = recommend_user_based(
        target_user=target_user,
        user_item_matrix=user_item_matrix,
        user_similarity=user_similarity,
        movies=movies,
        k=k_neighbors,
        top_n=top_k
    )

    recommended_items = set(recs["movieId"].tolist())
    true_positives = len(recommended_items.intersection(relevant_items))

    precision = true_positives / top_k if top_k > 0 else 0.0
    recall = true_positives / len(relevant_items) if len(relevant_items) > 0 else 0.0

    return precision, recall

def precision_recall_at_k_item_based(
    target_user: int,
    ratings: pd.DataFrame,
    user_item_matrix: pd.DataFrame,
```

```
    item_similarity: pd.DataFrame,
    movies: pd.DataFrame,
    k_neighbors: int = 20,
    top_k: int = 10,
    relevant_threshold: float = 4.0
):
    """
    Compute Precision@K and Recall@K for item-based CF.
    """
    user_true = ratings[ratings["userId"] == target_user]
    relevant_items = set(user_true[user_true["rating"] >=
relevant_threshold]["movieId"].tolist())

    if len(relevant_items) == 0:
        return 0.0, 0.0

    recs = recommend_item_based(
        target_user=target_user,
        user_item_matrix=user_item_matrix,
        item_similarity=item_similarity,
        movies=movies,
        k=k_neighbors,
        top_n=top_k
    )

    recommended_items = set(recs["movieId"].tolist())
    true_positives = len(recommended_items.intersection(relevant_items))
```

```
precision = true_positives / top_k if top_k > 0 else 0.0

recall = true_positives / len(relevant_items) if len(relevant_items) > 0 else 0.0


return precision, recall
```

Output

```
============================================================
Collaborative Filtering
============================================================

User-based CF recommendations:
   movieId                                              title                          genres  predicted_rating
0     5746                    Galaxy of Terror (Quest) (1981)  Action|Horror|Mystery|Sci-Fi               5.0
1     4495                            Crossing Delancey (1988)                Comedy|Romance               5.0
2     6201                                    Lady Jane (1986)                 Drama|Romance               5.0
3     3473  Jonah Who Will Be 25 in the Year 2000 (Jonas q...                        Comedy               5.0
4     6835                          Alien Contamination (1980)          Action|Horror|Sci-Fi               5.0
5    33649                                        Saving Face (2004)          Comedy|Drama|Romance               5.0
6     5088                Going Places (Valseuses, Les) (1974)            Comedy|Crime|Drama               5.0
7   118894                  Scooby-Doo! Abracadabra-Doo (2010)    Animation|Children|Mystery               5.0
8   136503            Tom and Jerry: Shiver Me Whiskers (2006)     Animation|Children|Comedy               5.0
9     5513              Martin Lawrence Live: Runteldat (2002)            Comedy|Documentary               5.0
```

```
Item-based CF recommendations:
   movieId                                              title                          genres  predicted_rating
0     6158                          Jungle Book 2, The (2003)              Animation|Children               5.0
1   193571                                   Silver Spoon (2014)                   Comedy|Drama               5.0
2    26366                          Harlan County U.S.A. (1976)                     Documentary               5.0
3   131724  The Jinx: The Life and Deaths of Robert Durst ...                     Documentary               5.0
4   193567  anohana: The Flower We Saw That Day - The Movi...                 Animation|Drama               5.0
5   158035  Gintama: The Final Chapter - Be Forever Yorozu...         Action|Animation|Comedy               5.0
6   193565                           Gintama: The Movie (2010)  Action|Animation|Comedy|Sci-Fi               5.0
7   191005                                     Gintama (2017)  Action|Adventure|Comedy|Sci-Fi               5.0
8   184641                     Fullmetal Alchemist 2018 (2017)        Action|Adventure|Fantasy               5.0
9   157369                                     Bakuman (2015)                     Comedy|Drama               5.0
```

5. Part 3 – Matrix Factorization

Matrix factorization techniques aim to **discover hidden relationships between users and items** by decomposing the rating matrix.

Task 5 – Singular Value Decomposition (SVD)

SVD decomposes the user-item matrix into three matrices:

$R \approx U \Sigma V^T$

Where:

- **U represents user latent features**
- **Σ represents singular values**
- **V represents item latent features**

These latent features capture hidden patterns such as **user preferences and movie characteristics**.

Predicted ratings are generated by reconstructing the matrix.

Task 6 – Matrix Factorization using Surprise Library

In this task, the **Surprise Python library** was used to implement SVD more efficiently.

The steps include:

1. Loading the dataset using Surprise.
2. Splitting the dataset into training and testing sets.
3. Training the **SVD model**.
4. Evaluating the model using **RMSE, Precision@K and Recall@K**.

This method simplifies the implementation of matrix factorization algorithms.

Code Implementation

```
import numpy as np
import pandas as pd
from sklearn.metrics import mean_squared_error


def prepare_svd_matrix(user_item_matrix: pd.DataFrame):
    """
    Fill missing values with user mean, then center by user mean.
    Returns:
        filled_matrix_df
        centered_matrix
        user_means
    """
    filled_matrix = user_item_matrix.copy()
```

```
    # Mean rating for each user
    user_means = filled_matrix.mean(axis=1)

    # Fill missing ratings with each user's mean rating
    filled_matrix = filled_matrix.apply(
        lambda row: row.fillna(row.mean()),
        axis=1
    )

    # If a user has all NaN values, fill with global mean fallback
    global_mean = user_item_matrix.stack().mean()
    filled_matrix = filled_matrix.fillna(global_mean)

    # Center rows by subtracting user mean
    centered_matrix = filled_matrix.sub(user_means.fillna(global_mean), axis=0)

    return filled_matrix, centered_matrix.values, user_means.fillna(global_mean)

def compute_svd(centered_matrix: np.ndarray, k: int = 20):
    """
    Perform truncated SVD using numpy.
    """
    U, s, Vt = np.linalg.svd(centered_matrix, full_matrices=False)

    U_k = U[:, :k]
    s_k = s[:k]
    Vt_k = Vt[:k, :]
```

```
    sigma_k = np.diag(s_k)

    return U_k, sigma_k, Vt_k

def reconstruct_ratings(
    U_k: np.ndarray,
    sigma_k: np.ndarray,
    Vt_k: np.ndarray,
    user_means: pd.Series
) -> np.ndarray:
    """
    Reconstruct approximate rating matrix and add back user means.
    """
    reconstructed_centered = U_k @ sigma_k @ Vt_k
    reconstructed = reconstructed_centered + user_means.values.reshape(-1, 1)

    # Clip to MovieLens rating scale
    reconstructed = np.clip(reconstructed, 0.5, 5.0)

    return reconstructed

def recommend_svd(
    target_user: int,
    user_item_matrix: pd.DataFrame,
    reconstructed_matrix: np.ndarray,
    movies: pd.DataFrame,
    top_n: int = 10
) -> pd.DataFrame:
```

```
    """
    Recommend top-N unseen movies for a user from reconstructed SVD ratings.
    """
    if target_user not in user_item_matrix.index:
        raise ValueError(f"User {target_user} not found.")

    user_idx = user_item_matrix.index.get_loc(target_user)
    predicted_ratings = reconstructed_matrix[user_idx]

    pred_df = pd.DataFrame({
        "movieId": user_item_matrix.columns,
        "predicted_rating": predicted_ratings
    })

    # Remove already rated movies
    rated_movies = user_item_matrix.loc[target_user].dropna().index.tolist()
    pred_df = pred_df[~pred_df["movieId"].isin(rated_movies)]

    pred_df = pred_df.sort_values(by="predicted_rating",
ascending=False).head(top_n)
    result = pred_df.merge(movies, on="movieId", how="left")

    return result[["movieId", "title", "genres",
"predicted_rating"]].reset_index(drop=True)

def evaluate_rmse_svd(
    ratings: pd.DataFrame,
    user_item_matrix: pd.DataFrame,
    reconstructed_matrix: np.ndarray,
```

```
    sample_size: int = 1000
) -> float:
    """
    Evaluate RMSE on a sample of known ratings.
    """
    sample = ratings.sample(n=min(sample_size, len(ratings)), random_state=42)

    y_true = []
    y_pred = []

    user_index_map = {uid: idx for idx, uid in enumerate(user_item_matrix.index)}
    movie_index_map = {mid: idx for idx, mid in
enumerate(user_item_matrix.columns)}

    for _, row in sample.iterrows():
        user_id = row["userId"]
        movie_id = row["movieId"]
        true_rating = row["rating"]

        if user_id in user_index_map and movie_id in movie_index_map:
            uidx = user_index_map[user_id]
            midx = movie_index_map[movie_id]
            pred = reconstructed_matrix[uidx, midx]

            y_true.append(true_rating)
            y_pred.append(pred)

    if len(y_true) == 0:
```

```
        return np.nan

    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    return float(rmse)

def precision_recall_at_k_svd(
    target_user: int,
    ratings: pd.DataFrame,
    user_item_matrix: pd.DataFrame,
    reconstructed_matrix: np.ndarray,
    movies: pd.DataFrame,
    top_k: int = 10,
    relevant_threshold: float = 4.0
):
    """
    Compute Precision@K and Recall@K for SVD recommender.
    """
    user_true = ratings[ratings["userId"] == target_user]
    relevant_items = set(user_true[user_true["rating"] >=
relevant_threshold]["movieId"].tolist())

    if len(relevant_items) == 0:
        return 0.0, 0.0

    recs = recommend_svd(
        target_user=target_user,
        user_item_matrix=user_item_matrix,
        reconstructed_matrix=reconstructed_matrix,
```

```
    movies=movies,
    top_n=top_k
)

recommended_items = set(recs["movieId"].tolist())
true_positives = len(recommended_items.intersection(relevant_items))

precision = true_positives / top_k if top_k > 0 else 0.0
recall = true_positives / len(relevant_items) if len(relevant_items) > 0 else 0.0

return precision, recall
```

Output

```
============================================================
Matrix Factorization (SVD)
============================================================
   movieId                                              title                        genres  predicted_rating
0      858                                Godfather, The (1972)                   Crime|Drama          4.663276
1      318                   Shawshank Redemption, The (1994)                   Crime|Drama          4.660012
2       32           Twelve Monkeys (a.k.a. 12 Monkeys) (1995)       Mystery|Sci-Fi|Thriller          4.582668
3      541                                Blade Runner (1982)        Action|Sci-Fi|Thriller          4.560365
4      778                               Trainspotting (1996)            Comedy|Crime|Drama          4.559684
5     1221                      Godfather: Part II, The (1974)                   Crime|Drama          4.548071
6     1288                           This Is Spinal Tap (1984)                         Comedy          4.536581
7      924                         2001: A Space Odyssey (1968)        Adventure|Drama|Sci-Fi          4.532593
8     7153  Lord of the Rings: The Return of the King, The...  Action|Adventure|Drama|Fantasy          4.529617
9     1199                                       Brazil (1985)                 Fantasy|Sci-Fi          4.529241
```

6. Hybrid Recommendation Model

A hybrid recommender combines **content-based filtering and collaborative filtering** to improve recommendation accuracy.

The final score is computed as:

Final Score = α × Content Score + (1 − α) × Collaborative Score

This approach benefits from the strengths of both techniques and helps **reduce the limitations of individual models**.

Code Implementation

```
import numpy as np
import pandas as pd
from sklearn.metrics import mean_squared_error
from sklearn.metrics.pairwise import cosine_similarity

def get_content_scores_for_user(
    user_id: int,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    tfidf_matrix
) -> pd.DataFrame:
    """
    Compute content-based similarity scores for all unseen movies
    for a given user using the user-profile method.
    """
    user_ratings = ratings[ratings["userId"] == user_id].copy()

    if user_ratings.empty:
        raise ValueError(f"User {user_id} not found.")

    movie_index_map = pd.Series(movies.index, index=movies["movieId"]).to_dict()
    user_ratings["movie_index"] = user_ratings["movieId"].map(movie_index_map)
    user_ratings = user_ratings.dropna(subset=["movie_index"]).copy()
    user_ratings["movie_index"] = user_ratings["movie_index"].astype(int)

    rated_indices = user_ratings["movie_index"].values
    weights = user_ratings["rating"].values.reshape(-1, 1)
```

```
    rated_movie_vectors = tfidf_matrix[rated_indices]
    weighted_profile = rated_movie_vectors.multiply(weights).sum(axis=0)

    total_rating = user_ratings["rating"].sum()
    user_profile = weighted_profile / total_rating

    user_profile = np.asarray(user_profile)
    similarity_scores = cosine_similarity(user_profile, tfidf_matrix).flatten()

    scores_df = movies.copy()
    scores_df["content_score"] = similarity_scores

    rated_movie_ids = set(user_ratings["movieId"].tolist())
    scores_df = scores_df[~scores_df["movieId"].isin(rated_movie_ids)].copy()

    return scores_df[["movieId", "title", "genres",
"content_score"]].reset_index(drop=True)

def get_item_cf_scores_for_user(
    target_user: int,
    user_item_matrix: pd.DataFrame,
    item_similarity: pd.DataFrame,
    movies: pd.DataFrame,
    k: int = 20
) -> pd.DataFrame:
    """
    Compute item-based CF predicted scores for all unseen movies.
```

```
    """
    if target_user not in user_item_matrix.index:
        raise ValueError(f"User {target_user} not found.")

    user_rated_movies = user_item_matrix.loc[target_user].dropna().index.tolist()
    unseen_movies = [m for m in user_item_matrix.columns if m not in
user_rated_movies]

    predictions = []
    user_ratings = user_item_matrix.loc[target_user].dropna()

    for movie_id in unseen_movies:
        if movie_id not in item_similarity.index:
            continue

        sims = item_similarity.loc[movie_id, user_rated_movies]
        top_k_items = sims.sort_values(ascending=False).head(k)

        neighbor_ratings = user_ratings.loc[top_k_items.index]
        denominator = np.abs(top_k_items).sum()

        if denominator == 0:
            continue

        predicted_rating = np.dot(top_k_items.values, neighbor_ratings.values) /
denominator
        predictions.append((movie_id, predicted_rating))

    pred_df = pd.DataFrame(predictions, columns=["movieId", "cf_score"])
```

```
    result = pred_df.merge(movies, on="movieId", how="left")

    return result[["movieId", "title", "genres", "cf_score"]].reset_index(drop=True)

def min_max_normalize(series: pd.Series) -> pd.Series:
    """
    Normalize values to [0,1].
    """
    min_val = series.min()
    max_val = series.max()

    if max_val == min_val:
        return pd.Series([0.0] * len(series), index=series.index)

    return (series - min_val) / (max_val - min_val)

def recommend_hybrid(
    user_id: int,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    tfidf_matrix,
    user_item_matrix: pd.DataFrame,
    item_similarity: pd.DataFrame,
    alpha: float = 0.5,
    k_cf: int = 20,
    top_n: int = 10
) -> pd.DataFrame:
    """
```

```
    Hybrid recommender combining content-based and item-based CF scores.
    """
    content_df = get_content_scores_for_user(
        user_id=user_id,
        ratings=ratings,
        movies=movies,
        tfidf_matrix=tfidf_matrix
    )

    cf_df = get_item_cf_scores_for_user(
        target_user=user_id,
        user_item_matrix=user_item_matrix,
        item_similarity=item_similarity,
        movies=movies,
        k=k_cf
    )

    merged = content_df.merge(
        cf_df[["movieId", "cf_score"]],
        on="movieId",
        how="inner"
    )

    if merged.empty:
        raise ValueError("No overlapping hybrid candidates found.")

    merged["content_score_norm"] =
min_max_normalize(merged["content_score"])
```

```
    merged["cf_score_norm"] = min_max_normalize(merged["cf_score"])

    merged["hybrid_score"] = (
        alpha * merged["content_score_norm"] +
        (1 - alpha) * merged["cf_score_norm"]
    )

    merged = merged.sort_values(by="hybrid_score", ascending=False).head(top_n)

    return merged[
        ["movieId", "title", "genres", "content_score", "cf_score", "hybrid_score"]
    ].reset_index(drop=True)

def evaluate_rmse_hybrid(
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    tfidf_matrix,
    user_item_matrix: pd.DataFrame,
    item_similarity: pd.DataFrame,
    alpha: float = 0.5,
    sample_size: int = 500
) -> float:
    """
    Simple hybrid RMSE evaluation on sampled known ratings.
    """
    sample = ratings.sample(n=min(sample_size, len(ratings)), random_state=42)

    y_true = []
```

```
    y_pred = []

    movie_index_map = pd.Series(movies.index, index=movies["movieId"]).to_dict()

    for _, row in sample.iterrows():
        user_id = row["userId"]
        movie_id = row["movieId"]
        true_rating = row["rating"]

        if user_id not in user_item_matrix.index or movie_id not in movie_index_map:
            continue

        # Content score
        try:
            user_ratings = ratings[ratings["userId"] == user_id].copy()
            user_ratings["movie_index"] =
user_ratings["movieId"].map(movie_index_map)
            user_ratings = user_ratings.dropna(subset=["movie_index"]).copy()
            user_ratings["movie_index"] = user_ratings["movie_index"].astype(int)

            rated_indices = user_ratings["movie_index"].values
            weights = user_ratings["rating"].values.reshape(-1, 1)
            rated_movie_vectors = tfidf_matrix[rated_indices]
            weighted_profile = rated_movie_vectors.multiply(weights).sum(axis=0)
            total_rating = user_ratings["rating"].sum()
            user_profile = weighted_profile / total_rating

            target_index = movie_index_map[movie_id]
```

```
        content_score = cosine_similarity(user_profile,
tfidf_matrix[target_index]).flatten()[0]
    except Exception:
        continue

    # CF score
    try:
        user_rated_movies = user_item_matrix.loc[user_id].dropna()
        rated_movie_ids = user_rated_movies.index.tolist()

        if movie_id not in item_similarity.index:
            continue

        sims = item_similarity.loc[movie_id, rated_movie_ids]
        top_k_items = sims.sort_values(ascending=False).head(20)
        neighbor_ratings = user_rated_movies.loc[top_k_items.index]
        denominator = np.abs(top_k_items).sum()

        if denominator == 0:
            continue

        cf_score = np.dot(top_k_items.values, neighbor_ratings.values) /
denominator
    except Exception:
        continue

    # Simple blend without normalization for single prediction
    pred = alpha * (content_score * 5.0) + (1 - alpha) * cf_score
    pred = float(np.clip(pred, 0.5, 5.0))
```

```
        y_true.append(true_rating)
        y_pred.append(pred)

    if len(y_true) == 0:
        return np.nan

    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    return float(rmse)

def precision_recall_at_k_hybrid(
    user_id: int,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    tfidf_matrix,
    user_item_matrix: pd.DataFrame,
    item_similarity: pd.DataFrame,
    alpha: float = 0.5,
    top_k: int = 10,
    relevant_threshold: float = 4.0
):
    """
    Compute Precision@K and Recall@K for the hybrid recommender.
    """
    relevant_items = set(
        ratings[
            (ratings["userId"] == user_id) &
            (ratings["rating"] >= relevant_threshold)
```

```
    ]["movieId"].tolist()
)

if len(relevant_items) == 0:
    return 0.0, 0.0

recs = recommend_hybrid(
    user_id=user_id,
    ratings=ratings,
    movies=movies,
    tfidf_matrix=tfidf_matrix,
    user_item_matrix=user_item_matrix,
    item_similarity=item_similarity,
    alpha=alpha,
    k_cf=20,
    top_n=top_k
)

recommended_items = set(recs["movieId"].tolist())
true_positives = len(recommended_items.intersection(relevant_items))

precision = true_positives / top_k if top_k > 0 else 0.0
recall = true_positives / len(relevant_items) if len(relevant_items) > 0 else 0.0

return precision, recall
```

Output

```
============================================================
Hybrid Recommendation
============================================================
   movieId                                              title                                       genres  content_score  cf_score  hybrid_s
core
0   117646                Dragonheart 2: A New Beginning (2000)   Action|Adventure|Comedy|Drama|Fantasy|Thriller       0.791026  4.719923      0.95
3321
1   164226                                Maximum Ride (2016)  Action|Adventure|Comedy|Fantasy|Sci-Fi|Thriller       0.759670  4.754760      0.93
9307
2   161594                Kingsglaive: Final Fantasy XV (2016)  Action|Adventure|Animation|Drama|Fantasy|Sci-Fi       0.683557  5.000000      0.93
2070
3   191005                                       Gintama (2017)                   Action|Adventure|Comedy|Sci-Fi       0.677407  5.000000      0.92
8182
4    55116                          Hunting Party, The (2007)           Action|Adventure|Comedy|Drama|Thriller       0.773645  4.630526      0.92
7434
5    65588                  Gamers, The: Dorkness Rising (2008)                  Action|Adventure|Comedy|Fantasy       0.707472  4.754760      0.90
6313
6   177285  Sword Art Online The Movie: Ordinal Scale (2017)        Action|Adventure|Animation|Fantasy|Sci-Fi       0.640243  5.000000      0.90
4692
7    48774                             Children of Men (2006)           Action|Adventure|Drama|Sci-Fi|Thriller       0.688794  4.806246      0.90
3088
8     6016                  City of God (Cidade de Deus) (2002)            Action|Adventure|Crime|Drama|Thriller       0.724926  4.666441      0.90
2625
9     7048                            Nothing to Lose (1997)                    Action|Adventure|Comedy|Crime       0.725346  4.662275      0.90
2196
```

7. Learning Based Recommender System

Advanced recommendation systems use **deep learning techniques** to learn complex user preferences.

Neural networks generate **user embeddings and movie embeddings**, which are used to predict ratings.

This approach allows the system to capture **non-linear relationships between users and items**.

Code Implementation

```
import re

import numpy as np

import pandas as pd

from sklearn.metrics import mean_squared_error

from sklearn.model_selection import train_test_split

from tensorflow.keras.layers import Input, Dense, Concatenate

from tensorflow.keras.models import Model

from tensorflow.keras.optimizers import Adam


def extract_year_from_title(title: str) -> int:

    """
```

```
    Extract release year from movie title like 'Toy Story (1995)'.
    """
    match = re.search(r"\((\d{4})\)", str(title))
    if match:
        return int(match.group(1))
    return 2000

def prepare_movie_features(movies: pd.DataFrame, ratings: pd.DataFrame):
    """
    Create movie-level metadata features:
    - one-hot genres
    - release year
    - average movie rating
    """
    movies = movies.copy()

    movies["year"] = movies["title"].apply(extract_year_from_title)

    avg_movie_rating = ratings.groupby("movieId")["rating"].mean().reset_index()
    avg_movie_rating.columns = ["movieId", "avg_movie_rating"]

    movies = movies.merge(avg_movie_rating, on="movieId", how="left")
    movies["avg_movie_rating"] =
movies["avg_movie_rating"].fillna(ratings["rating"].mean())

    genre_dummies = movies["genres"].str.get_dummies(sep="|")
    movie_features = pd.concat(
        [
```

```
        movies[["movieId", "title", "year", "avg_movie_rating"]],
        genre_dummies,
    ],
    axis=1,
)

return movie_features, genre_dummies.columns.tolist()


def prepare_user_features(ratings: pd.DataFrame, movies: pd.DataFrame,
genre_columns: list[str]):
    """
    Create user features as average rating per genre.
    """
    movie_genres = movies[["movieId", "genres"]].copy()
    genre_dummies = movie_genres["genres"].str.get_dummies(sep="|")
    movie_genres = pd.concat([movie_genres[["movieId"]], genre_dummies],
axis=1)

    merged = ratings.merge(movie_genres, on="movieId", how="left")

    user_feature_rows = []
    for user_id, group in merged.groupby("userId"):
        row = {"userId": user_id}

        for genre in genre_columns:
            genre_rated = group[group.get(genre, 0) == 1]
            if len(genre_rated) > 0:
                row[f"user_genre_{genre}"] = genre_rated["rating"].mean()
            else:
```

```
            row[f"user_genre_{genre}"] = ratings["rating"].mean()

        user_feature_rows.append(row)

    user_features = pd.DataFrame(user_feature_rows)
    return user_features

def build_training_data(ratings: pd.DataFrame, movie_features: pd.DataFrame,
user_features: pd.DataFrame):
    """
    Build merged dataset for neural model training.
    """
    data = ratings.merge(movie_features, on="movieId", how="left")
    data = data.merge(user_features, on="userId", how="left")
    return data

def get_feature_columns(movie_features: pd.DataFrame, user_features:
pd.DataFrame):
    """
    Return column lists for user and movie features.
    """
    movie_feature_cols = [col for col in movie_features.columns if col not in
["movieId", "title"]]
    user_feature_cols = [col for col in user_features.columns if col != "userId"]
    return user_feature_cols, movie_feature_cols

def build_neural_recommender(user_input_dim: int, movie_input_dim: int,
learning_rate: float = 0.001):
    """
```

```
    Build two-branch neural recommender model.
    """
    user_input = Input(shape=(user_input_dim,), name="user_input")
    x_user = Dense(64, activation="relu")(user_input)
    user_embedding = Dense(32, activation="relu",
name="user_embedding")(x_user)

    movie_input = Input(shape=(movie_input_dim,), name="movie_input")
    x_movie = Dense(64, activation="relu")(movie_input)
    movie_embedding = Dense(32, activation="relu",
name="movie_embedding")(x_movie)

    combined = Concatenate()([user_embedding, movie_embedding])
    x = Dense(64, activation="relu")(combined)
    x = Dense(32, activation="relu")(x)
    output = Dense(1, activation="linear", name="rating_prediction")(x)

    model = Model(inputs=[user_input, movie_input], outputs=output)
    model.compile(optimizer=Adam(learning_rate=learning_rate), loss="mse",
metrics=["mae"])

    return model

def train_neural_recommender(
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    epochs: int = 5,
    batch_size: int = 256,
    learning_rate: float = 0.001
```

```
):
    """
    Prepare features, train neural recommender, and return model artifacts.
    """
    movie_features, genre_columns = prepare_movie_features(movies, ratings)
    user_features = prepare_user_features(ratings, movies, genre_columns)

    full_data = build_training_data(ratings, movie_features, user_features)
    user_feature_cols, movie_feature_cols = get_feature_columns(movie_features,
user_features)

    X_user = full_data[user_feature_cols].values.astype("float32")
    X_movie = full_data[movie_feature_cols].values.astype("float32")
    y = full_data["rating"].values.astype("float32")

    X_user_train, X_user_test, X_movie_train, X_movie_test, y_train, y_test =
train_test_split(
        X_user, X_movie, y, test_size=0.2, random_state=42
    )

    model = build_neural_recommender(
        user_input_dim=X_user.shape[1],
        movie_input_dim=X_movie.shape[1],
        learning_rate=learning_rate,
    )

    history = model.fit(
        [X_user_train, X_movie_train],
        y_train,
```

```
        validation_data=([X_user_test, X_movie_test], y_test),
        epochs=epochs,
        batch_size=batch_size,
        verbose=1,
    )

    predictions = model.predict([X_user_test, X_movie_test], verbose=0).flatten()
    predictions = np.clip(predictions, 0.5, 5.0)

    rmse = np.sqrt(mean_squared_error(y_test, predictions))

    artifacts = {
        "model": model,
        "history": history,
        "rmse": float(rmse),
        "movie_features": movie_features,
        "user_features": user_features,
        "genre_columns": genre_columns,
        "user_feature_cols": user_feature_cols,
        "movie_feature_cols": movie_feature_cols,
        "full_data": full_data,
    }

    return artifacts

def recommend_neural_model(
    user_id: int,
    ratings: pd.DataFrame,
```

```
    movies: pd.DataFrame,
    artifacts: dict,
    top_n: int = 10
):
    """
    Generate top-N recommendations for a user using the trained neural model.
    """
    model = artifacts["model"]
    movie_features = artifacts["movie_features"]
    user_features = artifacts["user_features"]
    user_feature_cols = artifacts["user_feature_cols"]
    movie_feature_cols = artifacts["movie_feature_cols"]

    if user_id not in user_features["userId"].values:
        raise ValueError(f"User {user_id} not found in user features.")

    rated_movies = set(ratings.loc[ratings["userId"] == user_id, "movieId"].tolist())

    user_row = user_features[user_features["userId"] == user_id].iloc[0]
    user_vector = user_row[user_feature_cols].values.astype("float32").reshape(1, -
1)

    candidate_movies =
movie_features[~movie_features["movieId"].isin(rated_movies)].copy()

    movie_matrix = candidate_movies[movie_feature_cols].values.astype("float32")
    user_matrix = np.repeat(user_vector, repeats=len(candidate_movies), axis=0)

    predictions = model.predict([user_matrix, movie_matrix], verbose=0).flatten()
```

```
    predictions = np.clip(predictions, 0.5, 5.0)


    candidate_movies["predicted_rating"] = predictions
    candidate_movies = candidate_movies.sort_values(by="predicted_rating",
ascending=False).head(top_n)


    result = candidate_movies.merge(movies[["movieId", "title", "genres"]],
on="movieId", how="left")


    result = result.rename(columns={"title_x": "title", "title_y": "title_movie"})


    if "title" not in result.columns:
      if "title_movie" in result.columns:
        result["title"] = result["title_movie"]


    return result[["movieId", "title", "genres",
"predicted_rating"]].reset_index(drop=True)


def precision_recall_at_k_neural(
    user_id: int,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    artifacts: dict,
    top_k: int = 10,
    relevant_threshold: float = 4.0
):
    """
    Compute Precision@K and Recall@K for neural recommender.
    """
```

```
relevant_items = set(
    ratings[
        (ratings["userId"] == user_id) &
        (ratings["rating"] >= relevant_threshold)
    ]["movieId"].tolist()
)

if len(relevant_items) == 0:
    return 0.0, 0.0

recs = recommend_neural_model(
    user_id=user_id,
    ratings=ratings,
    movies=movies,
    artifacts=artifacts,
    top_n=top_k,
)

recommended_items = set(recs["movieId"].tolist())
true_positives = len(recommended_items.intersection(relevant_items))

precision = true_positives / top_k if top_k > 0 else 0.0
recall = true_positives / len(relevant_items) if len(relevant_items) > 0 else 0.0

return precision, recall
```

Output

```
============================================================
Neural Network Recommendation
============================================================
Epoch 1/3
316/316 ━━━━━━━━━━━━━━━━━━━━ 1s 870us/step - loss: 12.1644 - mae: 1.5182 - val_loss: 1.0620 - val_mae: 0.8401
Epoch 2/3
316/316 ━━━━━━━━━━━━━━━━━━━━ 0s 686us/step - loss: 0.9609 - mae: 0.7732 - val_loss: 0.9174 - val_mae: 0.7472
Epoch 3/3
316/316 ━━━━━━━━━━━━━━━━━━━━ 0s 688us/step - loss: 0.9089 - mae: 0.7452 - val_loss: 0.9008 - val_mae: 0.7447
   movieId                                      title                                 genres  predicted_rating
0    26875  Pure Formality, A (Pura formalità, Una) (1994)         Crime|Film-Noir|Mystery|Thriller          4.305523
1    32587                              Sin City (2005)  Action|Crime|Film-Noir|Mystery|Thriller          4.303917
2      164                   Devil in a Blue Dress (1995)         Crime|Film-Noir|Mystery|Thriller          4.302761
3     4037                          House of Games (1987)         Crime|Film-Noir|Mystery|Thriller          4.302732
4     1252                              Chinatown (1974)         Crime|Film-Noir|Mystery|Thriller          4.300888
5     4848                        Mulholland Drive (2001)   Crime|Drama|Film-Noir|Mystery|Thriller          4.293226
6     4298       Rififi (Du rififi chez les hommes) (1955)                 Crime|Film-Noir|Thriller          4.283968
7    58879                           Shine a Light (2008)                 Documentary|Musical|IMAX          4.282681
8     1248                           Touch of Evil (1958)                 Crime|Film-Noir|Thriller          4.282631
9    88129                                    Drive (2011)           Crime|Drama|Film-Noir|Thriller          4.281666
```

8. Reinforcement Learning Based Recommendation

Reinforcement learning treats recommendation as a **sequential decision-making problem**.

The system learns from user feedback using concepts such as:

- **State – User interaction history**
- **Action – Recommended movies**
- **Reward – User rating or feedback**

Algorithms such as **ε-Greedy and Q-Learning** help the system balance **exploration and exploitation**.

Code Implementation

```
import numpy as np

import pandas as pd


def build_q_table(ratings: pd.DataFrame):

    """

    Build Q-table initialized with zeros

    Rows = users

    Columns = movies

    """
```

```
    users = ratings["userId"].unique()
    movies = ratings["movieId"].unique()

    q_table = pd.DataFrame(
        0,
        index=users,
        columns=movies,
        dtype=float
    )

    return q_table

def train_q_learning(
    ratings: pd.DataFrame,
    episodes: int = 5,
    alpha: float = 0.1,
    gamma: float = 0.9
):
    """
    Train Q-learning recommender using ratings as reward
    """

    q_table = build_q_table(ratings)

    for episode in range(episodes):

        for _, row in ratings.iterrows():
```

```
        user = row["userId"]
        movie = row["movieId"]
        reward = row["rating"]

        current_q = q_table.loc[user, movie]

        max_future_q = q_table.loc[user].max()

        new_q = current_q + alpha * (
            reward + gamma * max_future_q - current_q
        )

        q_table.loc[user, movie] = new_q

    return q_table

def recommend_rl(
    user_id: int,
    q_table: pd.DataFrame,
    movies: pd.DataFrame,
    ratings: pd.DataFrame,
    top_n: int = 10
):
    """
    Recommend movies using learned Q-table
    """

    if user_id not in q_table.index:
```

```
        raise ValueError("User not found in Q-table")

    rated_movies = ratings[ratings["userId"] == user_id]["movieId"].tolist()

    user_q_values = q_table.loc[user_id].copy()

    user_q_values = user_q_values.drop(rated_movies, errors="ignore")

    top_movies = user_q_values.sort_values(ascending=False).head(top_n)

    result = pd.DataFrame({
        "movieId": top_movies.index,
        "q_score": top_movies.values
    })

    result = result.merge(
        movies[["movieId", "title", "genres"]],
        on="movieId",
        how="left"
    )

    return result[["movieId", "title", "genres", "q_score"]]

def evaluate_rl_rmse(
    ratings: pd.DataFrame,
    q_table: pd.DataFrame
):
    """
```

```
Evaluate RL predictions using RMSE
"""

predictions = []
actuals = []

for _, row in ratings.iterrows():

    user = row["userId"]
    movie = row["movieId"]

    if user in q_table.index and movie in q_table.columns:

        pred = q_table.loc[user, movie]

        predictions.append(pred)
        actuals.append(row["rating"])

predictions = np.array(predictions)
actuals = np.array(actuals)

rmse = np.sqrt(np.mean((predictions - actuals) ** 2))

return rmse
```

Output

```
============================================================
Reinforcement Learning Recommendation
============================================================
   movieId                                title                        genres  q_score
0      318     Shawshank Redemption, The (1994)                   Crime|Drama      0.0
1    91842                    Contraband (2012)    Action|Crime|Drama|Thriller      0.0
2   105197                      Nebraska (2013)               Adventure|Drama      0.0
3   105585     Machete Kills (Machete 2) (2013)          Action|Crime|Thriller      0.0
4   106441                Book Thief, The (2013)            Children|Drama|War      0.0
5   106785                     Homefront (2013)          Action|Crime|Thriller      0.0
6   109850                 Need for Speed (2014)        Action|Crime|Drama|IMAX      0.0
7   112818            Purge: Anarchy, The (2014)         Action|Horror|Thriller      0.0
8   112897              The Expendables 3 (2014)               Action|Adventure      0.0
9   114246  Walk Among the Tombstones, A (2014)  Action|Crime|Mystery|Thriller      0.0
```

9.Surprise_Model Recommender System

The Surprise model implementation uses the Singular Value Decomposition (SVD) algorithm from the Scikit-Surprise library to perform collaborative filtering on the MovieLens dataset. The ratings data is first converted into a format compatible with the Surprise framework using the Reader and Dataset classes. The dataset is then split into training and testing sets to evaluate the model's performance. The SVD algorithm learns latent factors representing user preferences and item characteristics, enabling the model to predict unknown ratings. After training, the model generates personalized movie recommendations by estimating ratings for movies a user has not yet watched and selecting the top-rated predictions. Model performance is evaluated using Root Mean Squared Error (RMSE) to measure prediction accuracy, along with Precision@K and Recall@K to assess recommendation quality. This approach helps capture hidden relationships between users and movies, improving recommendation accuracy compared to basic collaborative filtering methods.

Code Content

```
import numpy as np

import pandas as pd


def load_surprise_data(ratings: pd.DataFrame):
    """
    Convert pandas ratings dataframe into Surprise dataset.
    """
    from surprise import Dataset, Reader
```

```
    reader = Reader(rating_scale=(0.5, 5.0))
    data = Dataset.load_from_df(
        ratings[["userId", "movieId", "rating"]],
        reader
    )
    return data


def train_surprise_svd(
    ratings: pd.DataFrame,
    n_factors: int = 50,
    n_epochs: int = 20,
    lr_all: float = 0.005,
    reg_all: float = 0.02,
    test_size: float = 0.2,
    random_state: int = 42,
):
    """
    Train Surprise SVD and evaluate RMSE on a held-out test set.
    """
    from surprise import SVD
    from surprise.model_selection import train_test_split
    from surprise.accuracy import rmse

    data = load_surprise_data(ratings)
    trainset, testset = train_test_split(
        data,
        test_size=test_size,
```

```
        random_state=random_state
    )

    model = SVD(
        n_factors=n_factors,
        n_epochs=n_epochs,
        lr_all=lr_all,
        reg_all=reg_all,
        random_state=random_state,
    )

    model.fit(trainset)
    predictions = model.test(testset)
    test_rmse = rmse(predictions, verbose=False)

    return model, trainset, testset, predictions, float(test_rmse)


def get_surprise_top_n(
    model,
    ratings: pd.DataFrame,
    movies: pd.DataFrame,
    user_id: int,
    top_n: int = 10
) -> pd.DataFrame:
    """
    Recommend top-N unseen movies for a user using Surprise SVD.
    """
```

```
    rated_movie_ids = set(
        ratings.loc[ratings["userId"] == user_id, "movieId"].tolist()
    )

    all_movie_ids = movies["movieId"].tolist()

    predictions = []
    for movie_id in all_movie_ids:
        if movie_id not in rated_movie_ids:
            pred = model.predict(uid=user_id, iid=movie_id)
            predictions.append((movie_id, pred.est))

    pred_df = pd.DataFrame(predictions, columns=["movieId", "predicted_rating"])
    pred_df = pred_df.sort_values(by="predicted_rating",
ascending=False).head(top_n)

    result = pred_df.merge(
        movies[["movieId", "title", "genres"]],
        on="movieId",
        how="left"
    )

    return result[["movieId", "title", "genres",
"predicted_rating"]].reset_index(drop=True)


def precision_recall_at_k_surprise(
    predictions,
    k: int = 10,
```

```
    threshold: float = 4.0
):
    """
    Compute average Precision@K and Recall@K across users
    from Surprise prediction objects.
    """
    from collections import defaultdict

    user_est_true = defaultdict(list)

    for pred in predictions:
        uid = pred.uid
        est = pred.est
        true_r = pred.r_ui
        user_est_true[uid].append((est, true_r))

    precisions = {}
    recalls = {}

    for uid, user_ratings in user_est_true.items():
        user_ratings.sort(key=lambda x: x[0], reverse=True)

        n_rel = sum((true_r >= threshold) for (_, true_r) in user_ratings)
        top_k = user_ratings[:k]
        n_rec_k = sum((est >= threshold) for (est, _) in top_k)
        n_rel_and_rec_k = sum(
            ((true_r >= threshold) and (est >= threshold))
            for (est, true_r) in top_k
```

```
        )

        precisions[uid] = n_rel_and_rec_k / n_rec_k if n_rec_k != 0 else 0.0
        recalls[uid] = n_rel_and_rec_k / n_rel if n_rel != 0 else 0.0

    avg_precision = np.mean(list(precisions.values())) if precisions else 0.0
    avg_recall = np.mean(list(recalls.values())) if recalls else 0.0

    return float(avg_precision), float(avg_recall)
```

Output

```
============================================================
Building Surprise Library SVD
============================================================
Surprise SVD model trained successfully.

============================================================
Top Surprise Recommendations for User: 1
============================================================
   movieId                                              title                          genres  predicted_rating
0     2160                                 Rosemary's Baby (1968)          Drama|Horror|Thriller          5.000000
1    38061                            Kiss Kiss Bang Bang (2005)  Comedy|Crime|Mystery|Thriller          5.000000
2      318                      Shawshank Redemption, The (1994)                   Crime|Drama          5.000000
3     3275                           Boondock Saints, The (2000)    Action|Crime|Drama|Thriller          5.000000
4     1204                             Lawrence of Arabia (1962)            Adventure|Drama|War          5.000000
5     2324            Life Is Beautiful (La Vita è bella) (1997)       Comedy|Drama|Romance|War          5.000000
6     1172        Cinema Paradiso (Nuovo cinema Paradiso) (1989)                          Drama          5.000000
7      741            Ghost in the Shell (Kôkaku kidôtai) (1995)               Animation|Sci-Fi          5.000000
8     4973  Amelie (Fabuleux destin d'Amélie Poulain, Le) ...                 Comedy|Romance          4.987584
9    48516                                  Departed, The (2006)           Crime|Drama|Thriller          4.981134

============================================================
Evaluation for Surprise SVD
============================================================
RMSE        : 0.8775
Precision@10: 0.5686
Recall@10   : 0.2867
```

10. Explainability in Recommender Systems

Explainability improves transparency by explaining why a recommendation was made.

Examples include:

- **Feature-based explanations** – “Recommended because you like Sci-Fi movies.”
- **Neighbourhood explanations** – “Users similar to you liked this movie.”
- **Model-agnostic explanations** using **LIME or SHAP**.

Explainability increases **user trust and system transparency**.

Code

```
import pandas as pd
import numpy as np
import os

from src.data_loader import load_movielens_data
from src.content_model import (
    build_tfidf_model,
    recommend_movies_by_title,
    build_user_profile,
    recommend_movies_for_user_profile
)

from src.collaborative_model import (
    build_user_cf,
    recommend_user_cf,
    build_item_cf,
    recommend_item_cf
)

from src.svd_model import (
    build_svd_model,
    recommend_svd
)

from src.hybrid_model import recommend_hybrid

from src.deep_model import (
```

```
    train_neural_recommender,
    recommend_neural_model
)

from src.rl_model import (
    train_q_learning,
    recommend_rl
)

from src.explainability import (
    explain_content_recommendation,
    explain_cf_recommendation,
    explain_hybrid_recommendation,
    explain_neural_recommendation,
    explain_rl_recommendation
)

# ---------------------------------------------------
# Utility
# ---------------------------------------------------

def print_header(title):
    print("\n" + "=" * 60)
    print(title)
    print("=" * 60)

# ---------------------------------------------------
# Main Pipeline
```

```
# ----------------------------------------------------

def main():

    print_header("Movie Recommendation Project")

    # ----------------------------------------------------
    # Load dataset
    # ----------------------------------------------------

    movies, ratings, tags, links = load_movielens_data()

    print("\nDataset loaded successfully.\n")

    print("Movies shape  :", movies.shape)
    print("Ratings shape :", ratings.shape)
    print("Tags shape    :", tags.shape)
    print("Links shape   :", links.shape)

    sample_title = "Toy Story (1995)"
    sample_user_id = 1

    # ----------------------------------------------------
    # Content Based Recommender
    # ----------------------------------------------------

    print_header("Building TF-IDF Content-Based Recommender")
```

```
tfidf_matrix, tfidf_vectorizer = build_tfidf_model(movies)

movie_recs = recommend_movies_by_title(
    sample_title,
    movies,
    tfidf_matrix
)

print("\nTop Recommendations for Movie:", sample_title)
print(movie_recs)

# ---------------------------------------------------
# User Profile Content Recommender
# ---------------------------------------------------

print_header("Top Recommendations for User Profile")

user_profile = build_user_profile(
    sample_user_id,
    ratings,
    movies,
    tfidf_matrix
)

profile_recs = recommend_movies_for_user_profile(
    user_profile,
    movies,
    tfidf_matrix
```

```
)

print(profile_recs)

# ---------------------------------------------------
# User-Based Collaborative Filtering
# ---------------------------------------------------

print_header("Building User-Based Collaborative Filtering")

user_matrix, user_similarity = build_user_cf(ratings)

user_cf_recs = recommend_user_cf(
    sample_user_id,
    ratings,
    movies,
    user_matrix,
    user_similarity
)

print(user_cf_recs)

# ---------------------------------------------------
# Item-Based Collaborative Filtering
# ---------------------------------------------------

print_header("Building Item-Based Collaborative Filtering")
```

```
item_matrix, item_similarity = build_item_cf(ratings)

item_cf_recs = recommend_item_cf(
    sample_user_id,
    ratings,
    movies,
    item_matrix,
    item_similarity
)

print(item_cf_recs)

# ------------------------------------------------
# Matrix Factorization (SVD)
# ------------------------------------------------

print_header("Building Matrix Factorization with NumPy SVD")

svd_model = build_svd_model(ratings)

svd_recs = recommend_svd(
    sample_user_id,
    ratings,
    movies,
    svd_model
)

print(svd_recs)
```

```
# ----------------------------------------------------
# Hybrid Recommender
# ----------------------------------------------------

print_header("Building Hybrid Recommender")

hybrid_recs = recommend_hybrid(
    sample_user_id,
    ratings,
    movies,
    tfidf_matrix,
    user_matrix
)

print(hybrid_recs)

# ----------------------------------------------------
# Neural Network Recommender
# ----------------------------------------------------

print_header("Building Neural Network Recommender")

neural_model = train_neural_recommender(ratings)

neural_recs = recommend_neural_model(
    sample_user_id,
    neural_model,
```

```
    ratings,
    movies
)

print(neural_recs)

# ----------------------------------------------------
# Reinforcement Learning Recommender
# ----------------------------------------------------

print_header("Building Reinforcement Learning Recommender")

q_table = train_q_learning(ratings)

rl_recs = recommend_rl(
    sample_user_id,
    q_table,
    movies
)

print(rl_recs)

# ----------------------------------------------------
# Explainability Section
# ----------------------------------------------------

print_header("Explainability of Recommendations")
```

```
print("\nContent Explanation\n")

content_explanations = explain_content_recommendation(
    sample_title,
    movie_recs
)

print(content_explanations[0])

print("\nCollaborative Filtering Explanation\n")

cf_explanations = explain_cf_recommendation(
    sample_user_id,
    user_cf_recs
)

print(cf_explanations[0])

print("\nHybrid Model Explanation\n")

hybrid_explanations = explain_hybrid_recommendation(
    hybrid_recs
)

print(hybrid_explanations[0])

print("\nNeural Model Explanation\n")
```

```
    neural_explanations = explain_neural_recommendation(
        neural_recs
    )

    print(neural_explanations[0])

    print("\nReinforcement Learning Explanation\n")

    rl_explanations = explain_rl_recommendation(
        rl_recs
    )

    print(rl_explanations[0])

    print("\nProject setup is working correctly.")

# ----------------------------------------------------

if __name__ == "__main__":
    main()
```

Output

```
============================================================
Saving Metrics
============================================================
All outputs saved successfully.
Processed data folder      : /Users/niranjan/Downloads/MovieRecommendation/data/processed
Recommendations folder     : /Users/niranjan/Downloads/MovieRecommendation/outputs/recommendations
Figures folder             : /Users/niranjan/Downloads/MovieRecommendation/outputs/figures
Metrics folder             : /Users/niranjan/Downloads/MovieRecommendation/outputs/metrics
Models folder              : /Users/niranjan/Downloads/MovieRecommendation/outputs/models
(movie_rec_env) niranjan@Mac MovieRecommendation %
```

Conclusion

This project explored multiple recommendation techniques using the MovieLens dataset. Traditional recommendation methods such as **content-based filtering and collaborative filtering** were implemented and evaluated.

More advanced approaches such as **matrix factorization, hybrid models, neural networks, and reinforcement learning** were also explored to improve recommendation accuracy.

The experimental results show that **hybrid and learning-based recommendation models outperform traditional methods** because they capture both user behaviour and item characteristics more effectively.

Additionally, the integration of **explainable AI techniques improves the transparency and interpretability of recommendation systems**, making them more reliable for real-world applications.

Overall, this project demonstrates how modern machine learning techniques can be used to build **scalable, personalized, and intelligent recommendation systems**.

Git Link: [Github Movie-Reccomendation]